Evaluating Automated Threat Modeling: An Evaluation Framework for Multi-Agent Attack Tree Generation

Cristian Leo
Amazon Web Services
crisleoo@amazon.com

Anton Dykyi
Amazon Web Services
antondk@amazon.co.uk

Danny Cortegaca
Amazon Web Services
dicorteg@amazon.co.uk

Daniel Begimher
Amazon Web Services
dbbegimh@amazon.com

Abstract

Automated threat modeling promises to reduce the expertise barrier and time cost of security analysis, yet no standardized methodology exists for measuring the quality of machine-generated threat models. We present an evaluation framework for THREATFOREST, a multi-agent system that generates attack trees mapped to MITRE ATT&CK techniques. Our framework decomposes threat modeling quality into three independently measurable capabilities—threat statement generation, attack tree generation, and TTP matching—each with multi-dimensional scoring rubrics, automated structural metrics, and subject matter expert (SME) review protocols. The framework is implemented as a tracing infrastructure built on OpenTelemetry and Langfuse, capturing every agent interaction with structured metadata suitable for both human evaluation and automated optimization. We define 12 score dimensions across a standardized 5-point categorical scale, automated metrics for attack tree structure and phase coverage, and an export pipeline for curating ground truth datasets. We report baseline measurements across **[TODO: N]** domains and outline a roadmap for iterative prompt optimization using DSPy's MIPROv2, where SME-scored traces feed directly into reflective optimization cycles. To our knowledge, this is the first comprehensive evaluation framework for automated threat modeling, establishing reproducible baselines against which future systems can be measured.

1 Introduction

Threat modeling is a cornerstone of secure software development [6], requiring practitioners to systematically identify threats, analyze attack vectors, and propose mitigations. As cloud-native architectures grow in complexity, manual threat modeling becomes increasingly impractical—a single system may expose hundreds of attack surfaces across multiple services, each requiring domain-specific security expertise.

Large language models (LLMs) offer a path toward automating this process. Recent multi-agent systems can generate threat statements, construct attack trees [5], and map findings to frameworks like MITRE ATT&CK [4]. However, a fundamental question remains unanswered: *how do we measure whether automated threat models are any good?*

Unlike code generation—where correctness can be verified by compilation and test suites—threat modeling quality is inherently multi-dimensional and subjective. A generated attack tree may

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

be syntactically valid yet unrealistic, structurally complete yet missing critical attack phases, or technically accurate yet unhelpful for defenders. No existing benchmark or evaluation methodology addresses this challenge.

We identify three gaps in the current landscape:

No standardized evaluation dimensions. Existing work evaluates threat modeling tools informally, typically through case studies or user surveys. There is no agreed-upon set of quality dimensions, scoring scales, or evaluation protocols.

No separation of capabilities. Automated threat modeling involves distinct sub-tasks—generating threat statements, constructing attack trees, mapping to TTP frameworks—each requiring different evaluation criteria. Existing approaches conflate these into a single quality judgment.

No infrastructure for iterative improvement. Without structured data collection and scoring, there is no path from evaluation to optimization. Improving prompt quality requires ground truth datasets that do not yet exist.

We address these gaps with an evaluation framework for THREATFOREST, a multi-agent threat modeling system. Our contributions are:

1. **Multi-dimensional evaluation taxonomy.** We decompose threat modeling quality into three capabilities—threat statement generation, attack tree generation, and TTP matching—with 12 score dimensions across a standardized 5-point categorical scale.
2. **Hybrid evaluation methodology.** We combine automated structural metrics (node count, path coverage, phase detection, syntax validation) with SME scoring protocols, enabling both scalable measurement and expert validation.
3. **Tracing infrastructure.** We implement the framework as a production tracing system built on OpenTelemetry and Langfuse, capturing every agent interaction with structured metadata, scores, and export pipelines for dataset curation.
4. **Baseline measurements.** We report the first quantitative baselines for automated threat modeling quality across **[TODO: N]** diverse domains.
5. **Optimization roadmap.** We describe how the evaluation framework enables iterative prompt optimization via DSPy's MIPROv2 [2], where SME-scored traces feed directly into reflective optimization cycles.

2 Related Work

Threat modeling tools. Microsoft Threat Modeling Tool and OWASP Threat Dragon assist practitioners but automate little of the core reasoning. Recent LLM-based approaches generate threat statements from system descriptions **[TODO: cite]**, but evaluate outputs only through case studies or informal expert review.

Attack tree generation. Schneier [5] formalized attack trees as a threat analysis methodology. Automated generation has been explored through formal methods and graph-based approaches **[TODO: cite]**, but evaluation remains ad hoc—typically limited to structural validity checks without quality assessment.

Security AI evaluation. Benchmarks exist for offensive security capabilities **[TODO: cite PACEbench, CAIBench]** and security knowledge **[TODO: cite SecQA]**, but none address the evaluation of *defensive* threat modeling outputs. The closest work evaluates incident response agents **[TODO: cite SIR-Bench]**, which measures investigation quality rather than threat model quality.

LLM evaluation frameworks. General-purpose LLM evaluation has matured significantly, with frameworks like DSPy [1] enabling programmatic optimization through automated metrics. LLM-as-Judge approaches **[TODO: cite]** provide scalable evaluation but raise reliability concerns in specialized domains. Our framework combines automated metrics with structured SME review to address domain-specific evaluation needs.

Prompt optimization. MIPROv2 [2] jointly optimizes instructions and few-shot demonstrations through Bayesian search. We design our evaluation framework to produce data directly consumable by such optimizers, closing the loop between evaluation and improvement.

Gap. No prior work provides a systematic, multi-dimensional evaluation framework for automated threat modeling with structured data collection suitable for iterative optimization.

3 THREATFOREST System Overview

Before describing the evaluation framework, we briefly summarize the system under evaluation. THREATFOREST is a multi-agent threat modeling system built on the Strands agentic framework that processes project documentation into attack trees mapped to MITRE ATT&CK.

3.1 Multi-Agent Pipeline

The system employs five specialized agents executed sequentially:

1. **Context Analysis.** Extracts structured understanding of the target system from documentation, architecture diagrams, and deployment specifications.
2. **Information Extraction.** Parses threat specifications from multiple formats (ThreatComposer JSON, Markdown, YAML) into structured threat statements.
3. **Attack Tree Generation.** Produces Mermaid-formatted attack trees with hierarchical attack paths, color-coded by attack phase.
4. **TTP Mapping.** Maps attack tree nodes to MITRE ATT&CK techniques using SecureBERT2.0 [3] embeddings and cosine similarity search over a pre-computed index of 600+ techniques.
5. **Mitigation Synthesis.** Generates prioritized mitigation recommendations for identified techniques.

3.2 Evaluation Challenge

Each agent produces qualitatively different outputs requiring different evaluation criteria. The context analysis agent produces structured summaries; the attack tree agent produces graph structures; the TTP mapping agent produces ranked technique lists. A single quality score cannot capture this diversity, motivating the multi-dimensional framework described in Section 4.

4 Evaluation Framework

Our framework decomposes threat modeling evaluation into three independently measurable capabilities, each with automated metrics and SME scoring dimensions. The framework is implemented as a tracing infrastructure that captures evaluation data in production, enabling both retrospective analysis and forward-looking optimization.

4.1 Design Principles

1. **Capability separation.** Threat statement generation, attack tree generation, and TTP matching are evaluated independently with capability-specific metrics.
2. **Standardized scales.** All SME scores use a uniform 5-point categorical scale (Table 1) to reduce cognitive load and improve inter-rater consistency.
3. **Hybrid measurement.** Automated metrics provide scalable, deterministic baselines; SME scores capture quality dimensions that resist automation.
4. **Optimization-ready data.** Every evaluation record is structured for direct consumption by prompt optimization frameworks.

4.2 Capability 1: Threat Statement Evaluation

Threat statement evaluation addresses two distinct scenarios:

Generation mode. When the system generates new threats from context, we evaluate:

Table 1: Standardized 5-point categorical scoring scale used across all evaluation dimensions.

Category	Description	Numeric Value
Excellent	Exceptional quality, no issues	1.00
Good	Above average, minor issues	0.75
Acceptable	Meets minimum requirements	0.50
Poor	Below expectations, significant issues	0.25
Unacceptable	Fails to meet requirements	0.00

- *Overall quality* — holistic assessment of the generated threat set
- *Relevance to context* — whether threats match the application's technology stack, architecture, and deployment model
- *Completeness* — coverage across threat categories (STRIDE, CIA triad)
- *Technical accuracy* — realism of threat sources, actions, and impacts
- *Hallucination score* — absence of fabricated content

Validation mode. When the system processes existing threat statements, we additionally measure:

$$\text{hallucination\_rate} = 1.0 - \frac{|\text{invented threats}|}{|\text{total output threats}|} \tag{1}$$

$$\text{preservation\_score} = \frac{|\text{preserved threats}|}{|\text{total input threats}|} \tag{2}$$

These metrics detect whether the system faithfully reproduces provided threats or introduces hallucinated content—a critical failure mode for validation workflows.

4.3 Capability 2: Attack Tree Evaluation

Attack tree evaluation combines automated structural metrics with SME quality scores.

4.3.1 Automated Metrics

We compute four structural metrics directly from the generated Mermaid output:

- **Node count.** Total nodes in the attack tree, indicating complexity.
- **Path count.** Number of distinct root-to-leaf attack paths.
- **Max depth.** Longest path from root to leaf, indicating attack chain length.
- **Branching factor.** Average children per non-leaf node.

4.3.2 Phase Coverage

We detect MITRE ATT&CK phases present in the attack tree by matching node descriptions against a keyword dictionary covering 12 standard phases (reconnaissance through impact). The phase coverage score is:

$$\text{phase\_coverage} = \frac{|\text{detected phases}|}{|\text{expected phases}|} \tag{3}$$

where expected phases are determined by the threat context. Our keyword dictionary maps 130+ terms to the 12 ATT&CK tactics, enabling automated detection without requiring explicit phase labels in the generated output.

4.3.3 SME Score Dimensions

SMEs evaluate attack trees on six dimensions using the standardized scale:

Table 2: Attack tree SME evaluation dimensions.

Dimension	Description
Overall quality	Holistic assessment of the attack tree
Structural quality	Depth, branching, organization
Technical realism	Feasibility of attack techniques
Attack path logic	Logical progression from initial access to impact
Completeness	Coverage of attack vectors and phases
Actionability	Usefulness for defenders and security teams

Table 3: TTP mapping quality scale.

Category	Description	Value
Excellent	Precise, directly applicable mapping	1.00
Good	Relevant technique, minor mismatch	0.75
Acceptable	Broadly relevant technique	0.50
Poor	Tangentially related technique	0.25
No mapping	Incorrect or irrelevant technique	0.00

4.4 Capability 3: TTP Matching Evaluation

TTP matching evaluation measures how accurately the system maps attack tree nodes to MITRE ATT&CK techniques. SMEs rate each mapping on a specialized 5-point scale:

We report standard information retrieval metrics: Precision@K (fraction of top-K mappings rated Good or better) and Mean Reciprocal Rank (MRR).

4.5 Tracing Infrastructure

The evaluation framework is implemented as a production tracing system rather than an offline evaluation script. This design decision ensures that every THREATFOREST execution—whether in development, testing, or production—produces evaluation-ready data.

4.5.1 Architecture

The tracing infrastructure consists of six components:

1. **Interfaces** (`ITrace`, `ISpan`, `ITracingManager`) — abstract base classes enabling dependency injection between Langfuse-backed and no-op implementations.
2. **Tracing Manager** — singleton coordinator that creates traces with unique IDs and session grouping, attaching workflow metadata (model version, project path, timestamp).
3. **Score Definitions** — 12 score dimensions across three capabilities, each with type validation, range checking, and category enforcement.
4. **Score Config Registry** — manages Langfuse score configurations with idempotent registration, ensuring score definitions are synchronized with the tracing backend.
5. **Metrics Calculator** — automated computation of structural metrics, phase coverage, and technique extraction from attack tree output.
6. **Export Pipeline** — filters and exports scored traces to Langfuse Datasets, supporting filtering by trace type, review status, date range, and ground truth candidacy.

4.5.2 Resilient Design

The tracing system implements a resilient wrapper that catches and logs all Langfuse API errors without interrupting the threat modeling workflow. When Langfuse is unavailable, the system transparently falls back to no-op implementations, ensuring that tracing never degrades the user experience.

Table 4: Evaluation domains and characteristics.

Domain	Key Technologies	Regulatory Context
Healthcare Analytics	AWS, S3, Lambda, DynamoDB	HIPAA
IoT Device Management	MQTT, Edge Computing, OTA	IoT Security
E-commerce Platform	Microservices, Payment APIs	PCI-DSS
GenAI Chatbot	Bedrock, RAG, Vector DB	AI Safety
Connected Vehicle	V2X, Telematics, OBD-II	Automotive

4.5.3 Data Schema

Each trace captures:

- **Input context**: application name, technologies, architecture, deployment model
- **Agent outputs**: generated threats, attack trees, TTP mappings per stage
- **Generation metadata**: model version, prompt version, latency, token counts
- **SME scores**: per-dimension categorical ratings with free-text feedback
- **Automated metrics**: structural measurements and phase coverage

4.5.4 Export Pipeline

The export pipeline transforms scored traces into Langfuse Dataset items with `input`/`expected_output` pairs suitable for evaluation and optimization. Filters support:

- Trace type: `threat_statement`, `attack_tree`, `ttp_matching`
- Review status: `pending_review`, `reviewed`
- Date range and ground truth candidacy

This pipeline directly produces the training data format required by DSPy optimizers, closing the loop between evaluation and optimization.

5 Baseline Evaluation

We establish baseline measurements by running THREATFOREST across diverse domains and collecting both automated metrics and SME scores. These baselines serve as the reference point against which future optimizations will be measured.

5.1 Evaluation Domains

We evaluate across five domains selected for diversity in technology stack, regulatory requirements, and threat landscape:

5.2 Methodology

For each domain, we:

1. Run THREATFOREST with the default (unoptimized) prompts
2. Collect automated structural metrics from the generated attack trees
3. Have **[TODO: N]** SMEs independently score outputs using the framework dimensions
4. Compute inter-rater reliability across SME scores

5.3 Results

[TODO: Fill in after running baseline evaluation]

Table 5: Automated structural metrics across evaluation domains (baseline).

Domain	Nodes	Paths	Depth	Phase Cov.
Healthcare	[TODO:]	[TODO:]	[TODO:]	[TODO:]
IoT	[TODO:]	[TODO:]	[TODO:]	[TODO:]
E-commerce	[TODO:]	[TODO:]	[TODO:]	[TODO:]
GenAI Chatbot	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Connected Vehicle	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Average	[TODO:]	[TODO:]	[TODO:]	[TODO:]

Table 6: SME scores across evaluation dimensions (baseline, averaged across domains).

Dimension	Mean Score
Threat Statement Generation	
Overall quality	[TODO:]
Relevance to context	[TODO:]
Completeness	[TODO:]
Technical accuracy	[TODO:]
Hallucination score	[TODO:]
Attack Tree Generation	
Overall quality	[TODO:]
Structural quality	[TODO:]
Technical realism	[TODO:]
Attack path logic	[TODO:]
Completeness	[TODO:]
Actionability	[TODO:]
TTP Matching	
Mapping quality	[TODO:]

5.3.1 Automated Metrics

5.3.2 SME Scores

5.3.3 TTP Mapping Accuracy

6 Toward Iterative Optimization

A key design goal of the evaluation framework is to produce data that directly enables prompt optimization. In this section, we describe the optimization methodology that the framework supports and present preliminary results from an initial optimization experiment.

6.1 Optimization Loop

The evaluation framework enables a closed-loop optimization cycle:

1. **Generate.** Run THREATFOREST across evaluation domains, producing traced outputs.
2. **Score.** SMEs review outputs using the framework's scoring dimensions; automated metrics are computed in parallel.
3. **Export.** The export pipeline produces Langfuse Dataset items with input/expected_output pairs.
4. **Optimize.** DSPy's MIPROv2 [2] jointly optimizes prompt instructions and few-shot demonstrations using Bayesian search over the exported dataset.
5. **Evaluate.** The optimized prompt is evaluated against a held-out test set using the same framework metrics.
6. **Deploy.** If improved, the optimized prompt replaces the baseline; the cycle repeats.

Table 7: TTP mapping retrieval metrics (baseline).

Metric	Score
Precision@1	**[TODO:]**
Precision@3	**[TODO:]**
Precision@5	**[TODO:]**
MRR	**[TODO:]**
High-confidence mappings (> 0.7)	**[TODO: %]**

Table 8: MIPROv2 optimization configurations.

Mode	Trials	Candidates	Est. Duration
Light	5–10	5	5–10 min
Medium	15–20	10	30–60 min
Heavy	30+	15	1–2 hr

6.2 MIPROv2 Integration

We configure MIPROv2 with three optimization modes trading off thoroughness for compute cost:

The optimizer receives the evaluation framework's composite score as its objective function, with the weighted metric decomposition (syntax 30%, path coverage 30%, color coding 20%, logical flow 20%) providing gradient signal across quality dimensions.

6.3 Preliminary Experiment

We conducted an initial optimization experiment on attack tree generation using a synthetic dataset of 7 examples with a 5/2 train/test split:

The optimizer did not improve over baseline, which we attribute to the extremely small dataset size (7 examples, well below the 20+ recommended by DSPy documentation). This result underscores the importance of the evaluation framework's data collection infrastructure: meaningful optimization requires a critical mass of SME-scored examples that can only be accumulated through systematic evaluation.

6.4 Roadmap

With the evaluation framework in place, the path to optimization is:

1. **Baseline collection (this work).** Run THREATFOREST across all evaluation domains and collect SME scores using the framework.
2. **Dataset expansion.** Accumulate 50–100 SME-scored examples across domains via the tracing infrastructure.
3. **Evaluator calibration.** Train an automated evaluator (LLM-as-Judge) against SME scores, targeting correlation $r > 0.85$.
4. **Full optimization.** Run MIPROv2 in heavy mode with the expanded dataset.
5. **Multi-stage optimization.** Extend optimization beyond attack trees to threat statement generation and TTP matching prompts.

7 Discussion

7.1 Framework Design Decisions

Categorical vs. numeric scales. We chose a 5-point categorical scale over continuous numeric scores to reduce SME cognitive load and improve inter-rater reliability. Categorical labels ("excellent" through "unacceptable") provide clearer anchoring than arbitrary numeric ranges, while the fixed mapping to $[0, 1]$ preserves compatibility with optimization objectives.

Table 9: Preliminary optimization results (attack tree generation, $n = 7$).

Metric	Baseline	Optimized
Average Score	0.500	0.500
Success Rate	50.0%	50.0%

Production tracing vs. offline evaluation. Embedding evaluation in the production tracing system—rather than building a separate evaluation harness—ensures that every execution produces evaluation-ready data. This eliminates the common gap between "how the system runs" and "how the system is evaluated."

Capability separation. Evaluating threat statements, attack trees, and TTP mappings independently allows targeted optimization of each agent without confounding effects. A system may excel at threat identification but produce poor attack trees; unified scores would mask this.

7.2 Limitations

- **SME availability.** The framework requires security domain experts for scoring. Scaling evaluation beyond a small number of domains requires either more SMEs or a calibrated automated evaluator.
- **Keyword-based phase detection.** The automated phase coverage metric relies on keyword matching, which may miss novel phrasings or produce false positives for ambiguous terms.
- **Domain coverage.** Our baseline evaluation covers five domains. Broader coverage is needed to assess generalization.
- **Inter-rater reliability.** We have not yet measured agreement across multiple SMEs on the same outputs. This is planned as part of the baseline collection.
- **Optimization results.** The preliminary optimization experiment was inconclusive due to insufficient data. Meaningful optimization results require the larger dataset enabled by this framework.

7.3 Broader Impact

By establishing measurable baselines for automated threat modeling, this framework enables the security community to objectively compare tools and track progress. The open-source implementation and standardized scoring rubrics lower the barrier for other researchers to evaluate their own systems against the same criteria.

However, quantitative evaluation of threat models carries risks. Over-optimization for specific metrics may produce outputs that score well but miss novel threats outside the evaluation taxonomy. We recommend that automated evaluation always be complemented by periodic expert review of edge cases.

8 Conclusion

We presented an evaluation framework for automated threat modeling that decomposes quality into three independently measurable capabilities—threat statement generation, attack tree generation, and TTP matching—with 12 score dimensions, automated structural metrics, and SME review protocols. The framework is implemented as a production tracing infrastructure on OpenTelemetry and Langfuse, ensuring that every system execution produces evaluation-ready data.

Our key insight is that *evaluation infrastructure is a prerequisite for optimization*. Without standardized metrics, structured data collection, and export pipelines, there is no path from "the system works" to "the system improves." By designing the evaluation framework first, we establish the foundation for iterative prompt optimization via DSPy's MIPROv2, where each cycle of SME scoring produces training data for the next optimization round.

We report baseline measurements across **[TODO: N]** domains and outline a concrete roadmap from evaluation to optimization. To our knowledge, this is the first comprehensive evaluation framework

for automated threat modeling, and we release it as open source to enable reproducible research in this emerging area.

References

[1] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Modarressi, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In *ICLR*, 2024.

[2] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Li, B. Brber, V. Nair, K. Santhanam, and O. Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. *arXiv preprint arXiv:2406.11695*, 2024.

[3] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer. SecureBERT: A domain-specific language model for cybersecurity. In *International Conference on Security and Privacy in Communication Systems*, 2023.

[4] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas. MITRE ATT&CK: Design and philosophy. Technical report, The MITRE Corporation, 2018.

[5] B. Schneier. Attack trees. *Dr. Dobb's Journal*, 24(12):21–29, 1999.

[6] A. Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, 2014.

A Score Definitions Reference

Table 10 lists all 12 score dimensions defined in the evaluation framework.

Table 10: Complete score dimension reference.

Capability	Dimension	Type	Scale
Threat Statement	overall_quality	Categorical	5-point
Threat Statement	relevance_to_context	Categorical	5-point
Threat Statement	completeness	Categorical	5-point
Threat Statement	technical_accuracy	Categorical	5-point
Threat Statement	hallucination_score	Categorical	5-point
Attack Tree	overall_quality	Categorical	5-point
Attack Tree	structural_quality	Categorical	5-point
Attack Tree	technical_realism	Categorical	5-point
Attack Tree	attack_path_logic	Categorical	5-point
Attack Tree	completeness	Categorical	5-point
Attack Tree	actionability	Categorical	5-point
TTP Matching	mapping_quality	Categorical	5-point*

*TTP matching uses "no_mapping" instead of "unacceptable" as the lowest category.

B Phase Detection Keywords

The automated phase coverage metric matches attack tree node descriptions against keyword lists for 12 MITRE ATT&CK tactics. The dictionary contains 130+ terms. Representative examples:

- **Initial Access**: phishing, exploit public, drive-by, supply chain, valid accounts
- **Privilege Escalation**: escalate, elevate, admin, root, UAC bypass, token
- **Lateral Movement**: pivot, spread, pass the hash, RDP, SSH, remote service
- **Exfiltration**: exfiltrate, transfer, upload, data theft, steal data

C Tracing Data Schema

[TODO: Include the full JSON schema for evaluation records]